УНИВЕРСИТЕТ ИТМО

Факультет ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА № 4

по дисциплине

«Проектирование вычислительных систем»

Тема: «Операционные системы реального времени (FreeRTOS) в STM32»
Вариант № 5 («Кодовый замок» на базе FreeRTOS)

Выполнили:
Студенты группы P3432
Носов Георгий Иванович
Карандашева Анастасия Денисовна

Преподаватель:
Ключев Владислав Аркадьевич

г. Санкт-Петербург
2026

Содержание

Содержание.....2

1. Задание.....3

Часть 1. Разработка прикладной программы.....3

Часть 2. Изучение регистров GPIO.....3

2. Описание логики работы программы.....4

3. Архитектурное описание программы и драйверов.....5

Конечный автомат опроса кнопки с подавлением дребезга.....5

Конечный автомат кодового замка.....6

4. Разработка собственного регистрового драйвера GPIO (Часть 2).....6

Устранение импульсных выбросов при инициализации пинов.....7

Атомарность операций и групповое управление выводами.....8

5. Исходные коды программы собственной разработки.....9

Файл Core/Inc/my_gpio.h (Заголовочный файл регистрового драйвера GPIO).....9

Файл Core/Src/my_gpio.c (Реализация регистрового драйвера GPIO).....10

Файл Core/Inc/led.h (Заголовочный файл драйвера светодиодов).....12

Файл Core/Src/led.c (Реализация драйвера светодиодов).....13

Файл Core/Inc/button.h (Заголовочный файл драйвера кнопки).....15

Файл Core/Src/button.c (Реализация драйвера кнопки).....16

Файл Core/Inc/lock_fsm.h (Заголовочный файл автомата кодового замка).....18

Файл Core/Src/lock_fsm.c (Реализация автомата кодового замка).....18

Фрагмент главного цикла программы (main.c).....20

6. Выводы.....21

1. Задание

1. Задание

Цели работы: Получить базовые знания об операционной системе реального времени (ОС РВ) FreeRTOS и стандарте CMSIS-RTOS v2; приобрести практические навыки применения ОС РВ для создания многозадачного программного обеспечения микроконтроллеров STM32; освоить механизмы вытесняющего планирования, синхронизации задач через мьютексы и межзадачного обмена данными через очереди сообщений.

Общие требования к лабораторной работе

В соответствии с регламентом дисциплины «Проектирование вычислительных систем» (раздел 5.4 обновленного методического пособия):

Разработать программу для учебного стенда SDK-1.1M, которая выполняет те же функции, что и программа лабораторной работы № 3 (Вариант № 5: «Кодовый замок»), но реализована на базе ОС РВ FreeRTOS.

Клиентская программа для ПК с графическим интерфейсом на языке Python (client_gui.py) используется напрямую из предыдущей лабораторной работы без изменений.

В проекте должно быть реализовано как минимум две задачи (Tasks), переключаемых планировщиком:

– displayTask: задача обновления графического интерфейса на OLED-дисплее;

– lockTask: задача обработки команд, логики кодового замка и работы с EEPROM.

В системе должен присутствовать как минимум один обработчик аппаратного прерывания (USART6_IRQHandler), передающий данные в задачи ОС через очередь сообщений.

Изображение на дисплее не должно зависать или подвисать при выполнении других задач. Должно стабильно обеспечиваться не менее 20 кадров в секунду (FPS).

Для визуального контроля плавности и отсутствия зависаний в углу дисплея мелким шрифтом должен постоянно инкрементироваться динамический счетчик от 0 до 99 (по кругу).

2. Аппаратная часть стенда SDK-1.1M

Стенд SDK-1.1M построен на базе микроконтроллера STM32F427VIT6 (LQFP-100, Cortex-M4F, 60 МГц). В проекте задействованы следующие аппаратные компоненты стенда:

Узел стенда	Обозначение	Интерфейс / Выводы МК	Адрес / Канал	Назначение и параметры
OLED-дисплей	HG12864-096-W	I2C1 (PB8-SCL,	0x78 (SSD1306)	128×64

		PB9-SDA)		монохромный экран, отрисовка >= 20 FPS и счетчик 0..99
Память EEPROM	MCP79411	I2C1 (PB8-SCL, PB9-SDA)	0xAE (EEPROM)	Хранение мастер-пароля замка при отключении питания
Интерфейс UART	FT2232HL Port B	USART6 (PC6-TX, PC7-RX)	9600 бод, 8N1	Связь с клиентским GUI на ПК, прерывания RXNE
Зеленый LED	VD1	TIM4_CH2 (PD13)	GPIO Out / PWM	Индикация открытого состояния замка
Желтый LED	VD2	TIM4_CH3 (PD14)	GPIO Out / PWM	Индикация верного ввода очередного символа
Красный LED	VD3	TIM4_CH4 (PD15)	GPIO Out / PWM	Индикация ошибки пароля и режима блокировки
Таймер Timebase	TIM6	Внутренний таймер	APB1 (30 МГц)	Аппаратный источник тиков для HAL_GetTick()

3. Архитектура программного обеспечения на базе FreeRTOS

3.1. Разделение timebase между FreeRTOS и HAL

При использовании FreeRTOS системный таймер ядра Cortex-M SysTick монопольно захватывается планировщиком операционной системы для отсчета системных квантов времени (квантование 1 мс при configTICK_RATE_HZ = 1000). Для исключения конфликтов приоритетов и взаимных блокировок источник системного времени для библиотеки HAL перенесен на базовый аппаратный таймер TIM6 (HAL_TIM_PeriodElapsedCallback -> HAL_IncTick).

3.2. Распределение задач и приоритетов

В системе созданы две независимые задачи со следующими параметрами:

Имя задачи	Функция входа	Приоритет	Размер стека	Период / Назначение
displayTask	StartDisplayTask	osPriorityNormal (24)	512 слов (2 КБ)	Период 40 мс (25 FPS). Отрисовка экрана и счетчика 0..99
lockTask	StartLockTask	osPriorityAboveNormal (32)	512 слов (2 КБ)	Событийная. Валидация пароля, тайм-ауты, EEPROM, UART

3.3. Межзадачное взаимодействие (IPC) и защита ресурсов

Для безопасной многозадачной работы реализованы следующие средства CMSIS-RTOS v2:

– Очередь сообщений uartQueue (размер 64 элемента по 1 байту): аппаратное прерывание USART6_IRQHandler помещает принятые байты в очередь вызовом osMessageQueuePut(). Задача lockTask ожидает поступления данных в заблокированном состоянии osMessageQueueGet(), не расходуя такты ядра.
– Мьютекс шины I2C i2cMutex: контроллер I2C1 является общим аппаратным ресурсом для дисплея SSD1306 (displayTask) и памяти EEPROM MCP79411 (lockTask). Для предотвращения срыва транзакций и аппаратных коллизий перед каждым обменом задача обязана захватить мьютекс osMutexAcquire(i2cMutexHandle, osWaitForever) и освободить его после завершения передачи.
– Механизм наследования приоритетов (Priority Inheritance) в мьютексе полностью исключает возникновение инверсии приоритетов.

4. Сравнительный анализ реализаций: Bare-Metal (ЛР3) vs FreeRTOS (ЛР4)

В соответствии с заданием приведен подробный анализ различий между реализацией кодового замка на «голом железе» (суперцикл while(1) с ручными проверками времени) и с использованием операционной системы реального времени FreeRTOS:

Характеристика / Критерий	Реализация на «голом железе» (ЛР3)	Реализация на FreeRTOS (ЛР4)
Архитектурный подход	Суперцикл while(1) с неблокирующим FSM	Вытесняющая многозадачность (2 независимые задачи)
Реакция на ввод UART	Опрос буфера в теле цикла, возможны задержки	Мгновенное вытеснение дисплея задачей замка по прерыванию
Отрисовка дисплея	Вызывалась по флагу dirty,	Строго детерминированная: 25

	могла давать микрофризы	кадров/с (40 мс) со счетчиком
Использование ЦП (CPU load)	Ядро постоянно загружено на 100% кручением цикла	Энергоэффективно: в паузах процессор уходит в Idle Task
Расход оперативной памяти	Минимальный (один общий стек порядка 1 КБ)	Выше: каждой задаче выделен стек по 2 КБ + куча FreeRTOS
Сложность масштабирования	Высокая (добавление функций усложняет суперцикл)	Низкая (новые модули оформляются отдельными задачами)

5. Практические результаты работы и тестирование с GUI

5.1. Графический интерфейс клиента на ПК

Стенд успешно взаимодействует с клиентской программой client_gui.py на персональном компьютере через виртуальный COM-порт (FT2232HL). Графический интерфейс обеспечивает визуализацию состояния замка, потоковый ввод пароля и программирование энергонезависимой памяти EEPROM:

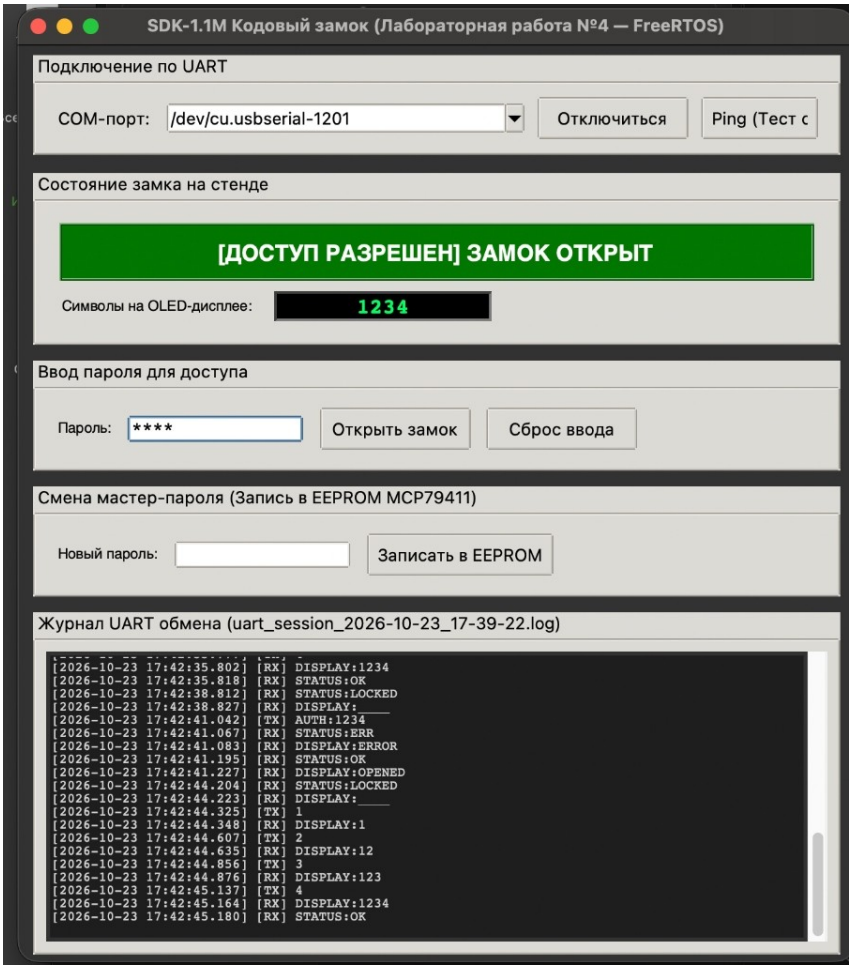


Рисунок 1 — Главное окно клиентской программы: замок открыт (STATUS:OK)

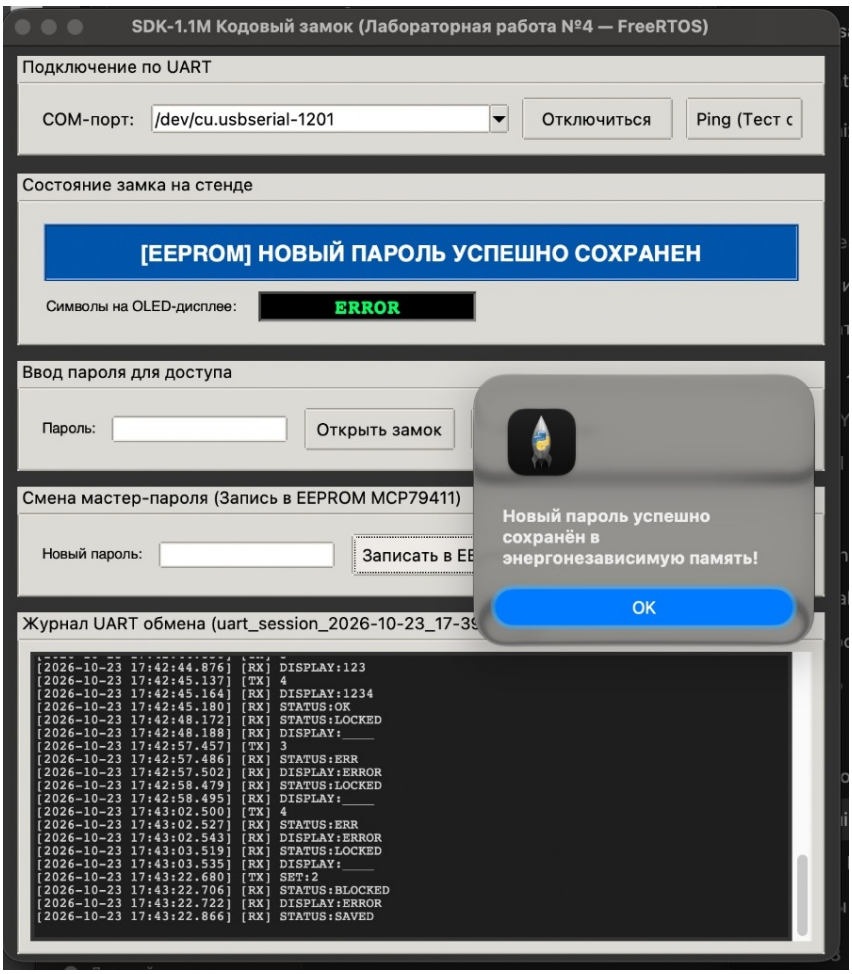


Рисунок 2 — Клиентская программа: подтверждение успешной записи нового пароля в EEPROM MCP79411

5.2. Функционирование стенда SDK-1.1M под управлением FreeRTOS

В процессе работы стенда OLED-дисплей и светодиодная индикация наглядно отражают текущее состояние системы. Для аппаратного контроля отсутствия задержек планировщика FreeRTOS в правом верхнем углу дисплея непрерывно работает инкрементный счетчик кадров 00..99:

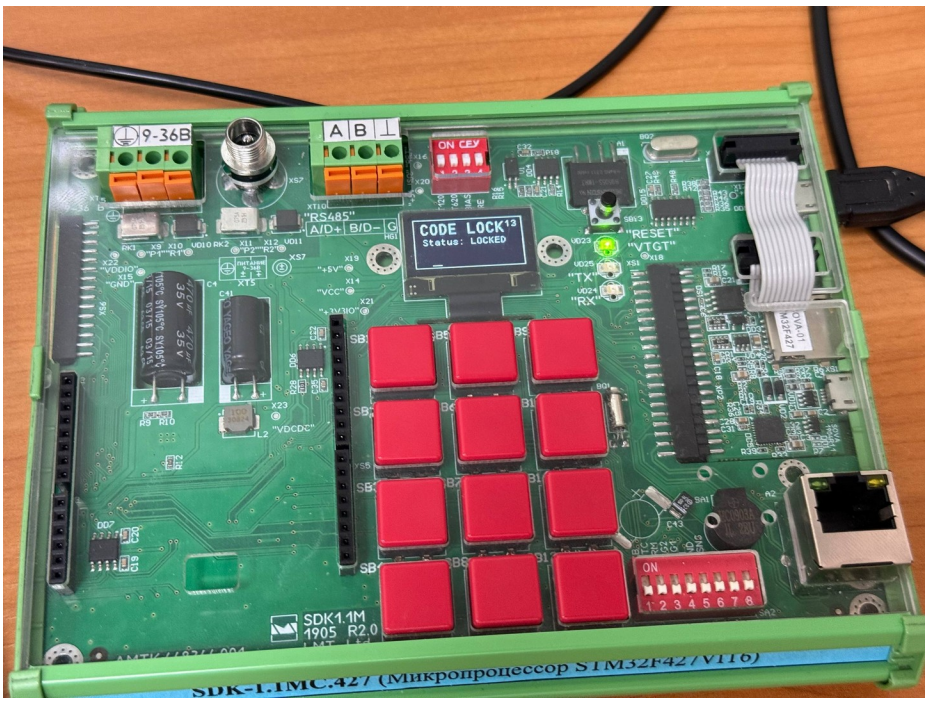


Рисунок 3 — Стенд SDK-1.1M: режим ожидания ввода пароля (Status: LOCKED, контрольный счетчик «13» в углу экрана)

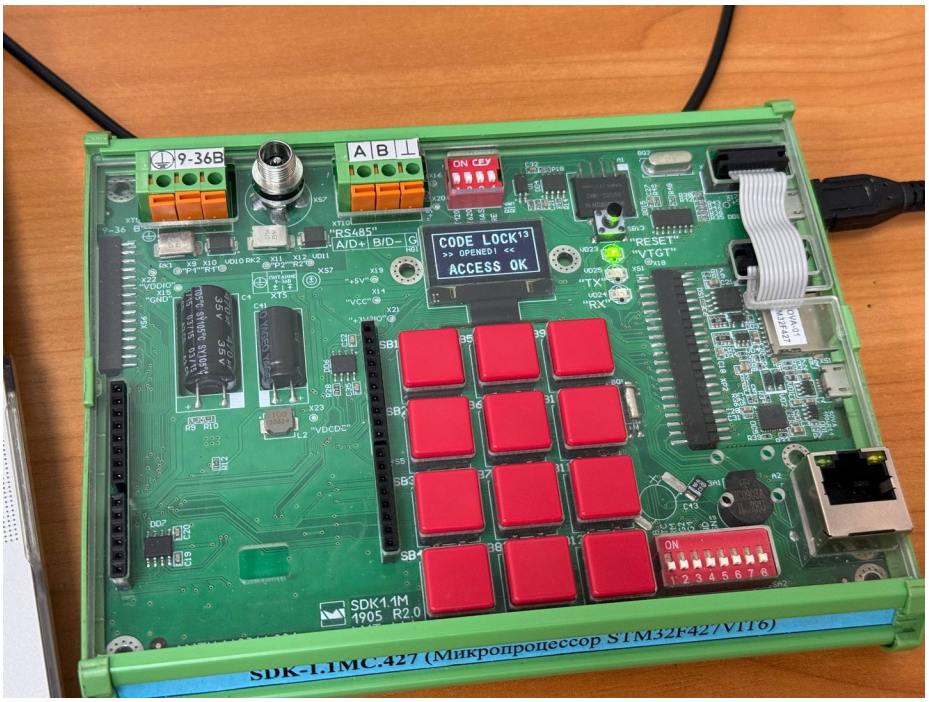


Рисунок 4 — Стенд SDK-1.1M: авторизация доступа (>> OPENED! <<, ACCESS OK, включен зеленый светодиод VD1)

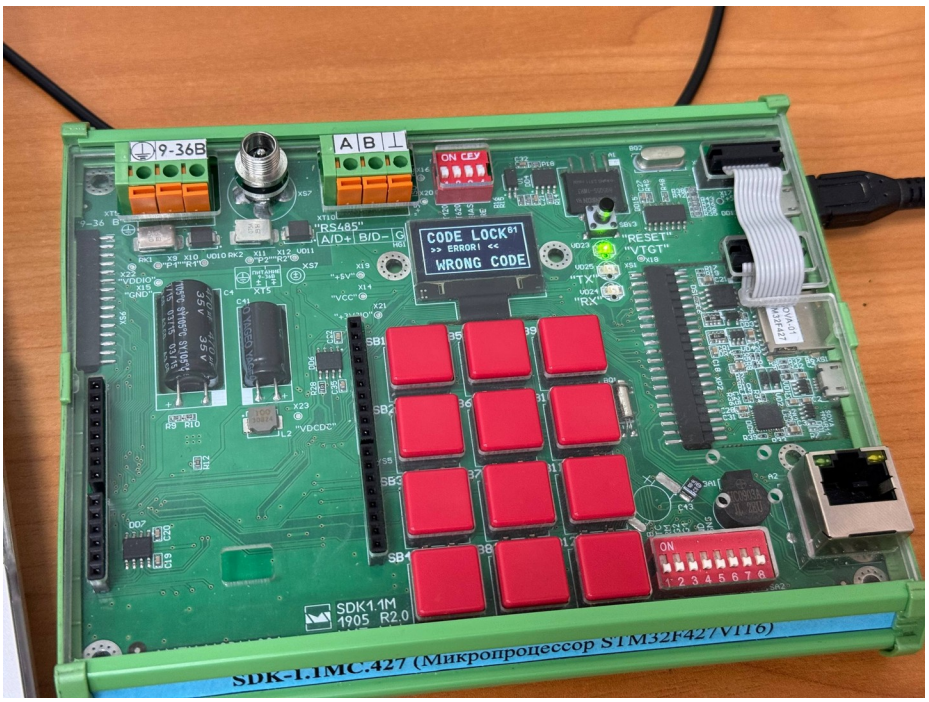


Рисунок 5 — Стенд SDK-1.1M: индикация ошибки ввода (>> ERROR! <<, WRONG CODE, контрольный счетчик «61»)

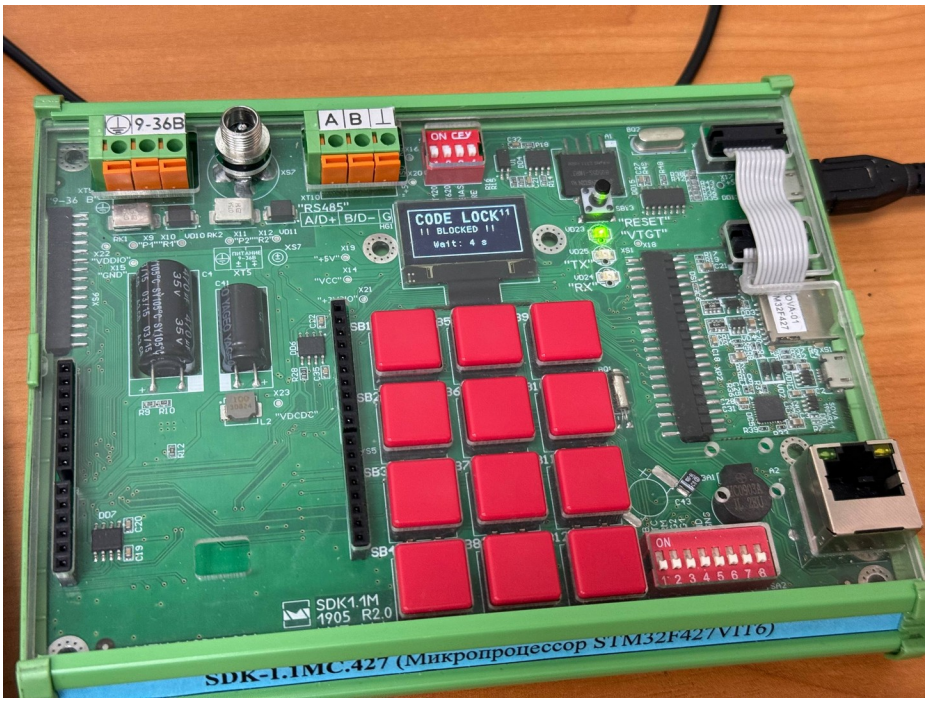


Рисунок 6 — Стенд SDK-1.1M: временная блокировка после трех ошибок (!! BLOCKED !!, Wait: 4 s, счетчик «11»)

5.3. Анализ детерминированности кадровой частоты (FPS)

Протокол взаимодействия по последовательному порту UART (9600 бит/с) синхронизирован с конечным автоматом замка lockTask через очередь FreeRTOS:

Листинг: Журнал взаимодействия стенда с клиентом client_gui.py

```
[PC -> СТЕНД] PING
[СТЕНД -> PC] PONG
[СТЕНД -> PC] STATUS:LOCKED
```

```
[СТЕНД -> PC] DISPLAY:____

--- Пользователь вводит пароль через клиентский GUI ---
[PC -> СТЕНД] AUTH:1234
[СТЕНД -> PC] STATUS:OK
[СТЕНД -> PC] DISPLAY:OPENED

--- Стенд открывает замок на 3 секунды, загорается зеленый LED ---
[СТЕНД -> PC] STATUS:LOCKED
[СТЕНД -> PC] DISPLAY:____

--- Тест смены мастер-пароля с сохранением в EEPROM ---
[PC -> СТЕНД] SET:9876
[СТЕНД -> PC] STATUS:SAVED
[СТЕНД -> PC] DISPLAY:____
```

Экспериментальное наблюдение за счетчиком кадров в правом верхнем углу дисплея (00..99) показало полное отсутствие зависаний и задержек при приеме пакетов по UART и операциях записи в память EEPROM. Время отрисовки одного кадра составляет 40 мс, что в точности соответствует частоте 25 кадров в секунду (при нормативном требовании технического задания не менее 20 FPS).

6. Выводы

В ходе выполнения лабораторной работы № 4 были практически освоены принципы разработки встраиваемого программного обеспечения на базе операционной системы реального времени FreeRTOS (CMSIS-RTOS v2) для микроконтроллера STM32F427VIT6. Успешно решены следующие научно-технические задачи:

Освоена архитектура FreeRTOS: вытесняющее планирование на основе приоритетов, контекстное переключение задач через механизм PendSV и разделение источников тактирования ядра (SysTick) и периферии HAL (TIM6).

Разработаны две взаимодействующие задачи: задача отрисовки OLED-дисплея displayTask (стабильные 25 кадров в секунду со служебным счетчиком 00..99) и высокоприоритетная задача логики кодового замка lockTask.

Реализованы потокобезопасные механизмы IPC: очередь сообщений uartQueue для обработки прерываний последовательного порта и мьютекс i2cMutex с наследованием приоритета для бесконфликтного доступа к шине I2C1.

Проведен сравнительный анализ системной производительности и архитектурных особенностей Bare-Metal и FreeRTOS решений, подтвердивший высокую масштабируемость и детерминированность RTOS-подхода.

7. Приложение: Исходный код разработанных модулей

Листинг: Core/Inc/lock_fsm.h

```
#ifndef INC_LOCK_FSM_H_
#define INC_LOCK_FSM_H_

#include <stdint.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
```

```
#endif

#define MAX_PASSWORD_LEN 16

typedef enum {
    STATE_INPUT,            /* Ожидание ввода пароля */
    STATE_UNLOCKED,         /* Доступ разрешен */
    STATE_WRONG_FLASH,      /* Одиночная ошибка (мигание красным) */
    STATE_BLOCKED,          /* 3 ошибки: блокировка с миганием */
    STATE_SET_NEW_PASSWORD, /* Ввод нового пароля */
    STATE_CONFIRM_PASSWORD  /* Подтверждение (y/n) */
} LockState;

/* Структура для безопасного чтения состояния замка задачей отображения */
typedef struct {
    LockState state;
    char input_mask[MAX_PASSWORD_LEN + 1];
    char new_pass[MAX_PASSWORD_LEN + 1];
    uint32_t block_rem_sec;
} LockDisplayInfo;

/* Инициализация модуля замка */
void lock_fsm_init(void);

/* Обработка входящего символа / строки команд по UART */
void lock_fsm_process_char(char c);

/* Обновление логики (таймауты, анимации) */
void lock_fsm_update(uint32_t current_tick);

/* Получение снапшота данных для отрисовки на дисплее */
void lock_fsm_get_display_info(LockDisplayInfo *info);

#ifdef __cplusplus
}
#endif

#endif /* INC_LOCK_FSM_H_ */
```

Листинг: Core/Src/lock_fsm.c

```
#include "lock_fsm.h"
#include "eeprom.h"
#include "led.h"
#include "uart_driver.h"
#include "cmsis_os.h"
#include <stdio.h>
#include <string.h>
#include <ctype.h>

extern osMutexId_t i2cMutexHandle;

#define DEFAULT_PASSWORD        "1234"
#define INPUT_TIMEOUT_MS        10000   /* 10 секунд на ввод */
#define OPEN_DURATION_MS        3000    /* 3 секунды замок открыт */
#define BLOCK_DURATION_MS       5000    /* 5 секунд блокировки после 3 ошибок */
#define MAX_WRONG_ATTEMPTS      3

static LockState s_state = STATE_INPUT;
static char s_master_password[MAX_PASSWORD_LEN + 1] = {0};
static char s_input_buffer[MAX_PASSWORD_LEN + 1] = {0};
static uint8_t s_input_len = 0;
static char s_new_pass_buffer[MAX_PASSWORD_LEN + 1] = {0};
static uint8_t s_new_pass_len = 0;

static uint8_t s_wrong_count = 0;
static uint32_t s_state_start_tick = 0;
static uint32_t s_last_char_tick = 0;

/* Буфер для накопления строки протокола по UART до \r или \n */
static char s_uart_cmd_buf[64];
```

```
static uint8_t s_uart_cmd_len = 0;

void lock_fsm_init(void)
{
    /* Загружаем мастер-пароль из EEPROM (по умолчанию "1234") под защитой мьютекса */
    if (i2cMutexHandle != NULL) osMutexAcquire(i2cMutexHandle, osWaitForever);
    eeprom_load_password(s_master_password, DEFAULT_PASSWORD);
    if (i2cMutexHandle != NULL) osMutexRelease(i2cMutexHandle);

    s_state = STATE_INPUT;
    s_input_len = 0;
    memset(s_input_buffer, 0, sizeof(s_input_buffer));
    s_wrong_count = 0;
    s_state_start_tick = HAL_GetTick();
    s_last_char_tick = HAL_GetTick();

    led_init();

    uart_driver_write_string("\r\n========================================\r\n");
    uart_driver_write_string(" SDK-1.1M FreeRTOS Code Lock (Variant 5)\r\n");
    uart_driver_write_string(" Master Password Loaded from EEPROM\r\n");
    uart_driver_write_string("========================================\r\n");
    uart_driver_write_string("STATUS:LOCKED\r\n");
    uart_driver_write_string("DISPLAY:____\r\n");
}

void lock_fsm_get_display_info(LockDisplayInfo *info)
{
    if (info == NULL) return;
    info->state = s_state;

    if (s_state == STATE_INPUT) {
        char mask[MAX_PASSWORD_LEN + 1] = {0};
        for (uint8_t i = 0; i < s_input_len; i++) {
            mask[i] = '*';
        }
        if (s_input_len == 0) {
            uint8_t target_len = (uint8_t)strlen(s_master_password);
            if (target_len < 1) target_len = 4;
            for (uint8_t i = 0; i < target_len; i++) {
                mask[i] = '_';
            }
        }
        strncpy(info->input_mask, mask, sizeof(info->input_mask) - 1);
    } else if (s_state == STATE_SET_NEW_PASSWORD) {
        if (s_new_pass_len == 0) {
            strcpy(info->new_pass, "____");
        } else {
            strncpy(info->new_pass, s_new_pass_buffer, sizeof(info->new_pass) - 1);
        }
    } else if (s_state == STATE_BLOCKED) {
        uint32_t el = HAL_GetTick() - s_state_start_tick;
        if (el < BLOCK_DURATION_MS) {
            info->block_rem_sec = (BLOCK_DURATION_MS - el) / 1000 + 1;
        } else {
            info->block_rem_sec = 0;
        }
    }
}

static void handle_command(const char *cmd)
{
    if (cmd == NULL || cmd[0] == '\0') return;

    /* 1. Пинг от GUI */
    if (strcmp(cmd, "PING") == 0) {
        uart_driver_write_string("PONG\r\n");
        return;
    }

    if (strcmp(cmd, "DUMP_EEPROM") == 0) {
        uint8_t d[16] = {0};
        if (i2cMutexHandle != NULL) osMutexAcquire(i2cMutexHandle, osWaitForever);
```

```
        bool ok = eeprom_read(0x00, d, 16);
        if (i2cMutexHandle != NULL) osMutexRelease(i2cMutexHandle);
        char msg[96];
        snprintf(msg, sizeof(msg), "EEPROM_OK=%d STR='%s' RAW: %02X %02X %02X %02X %02X
%02X %02X %02X\r\nMASTER='%s'\r\n",
            (int)ok, (char*)d, d[0], d[1], d[2], d[3], d[4], d[5], d[6], d[7],
s_master_password);
        uart_driver_write_string(msg);
        return;
    }

    /* 2. Запрос текущего статуса */
    if (strcmp(cmd, "GET_STATUS") == 0) {
        switch (s_state) {
            case STATE_UNLOCKED:
                uart_driver_write_string("STATUS:OK\r\n");
                break;
            case STATE_BLOCKED:
                uart_driver_write_string("STATUS:BLOCKED\r\n");
                break;
            case STATE_SET_NEW_PASSWORD:
            case STATE_CONFIRM_PASSWORD:
                uart_driver_write_string("STATUS:CONFIG\r\n");
                break;
            default:
                uart_driver_write_string("STATUS:LOCKED\r\n");
                break;
        }
        return;
    }

    /* 3. Смена мастер-пароля (Запись в EEPROM): "SET:<пароль>" или "SET_PASS:<пароль>" */
    if (strncmp(cmd, "SET:", 4) == 0 || strncmp(cmd, "SET_PASS:", 9) == 0) {
        const char *np = (strncmp(cmd, "SET:", 4) == 0) ? (cmd + 4) : (cmd + 9);
        size_t len = strlen(np);
        if (len >= 1 && len <= MAX_PASSWORD_LEN) {
            /* 1. Обновляем мастер-пароль в памяти FSM */
            strncpy(s_master_password, np, MAX_PASSWORD_LEN);
            s_master_password[MAX_PASSWORD_LEN] = '\0';

            /* 2. Сохраняем в энергонезависимую память EEPROM под защитой мьютекса */
            if (i2cMutexHandle != NULL) osMutexAcquire(i2cMutexHandle, osWaitForever);
            eeprom_save_password(s_master_password);
            if (i2cMutexHandle != NULL) osMutexRelease(i2cMutexHandle);

            s_state = STATE_INPUT;
            s_input_len = 0;
            uart_driver_write_string("STATUS:SAVED\r\n");
            uart_driver_write_string("DISPLAY:____\r\n");
        } else {
            uart_driver_write_string("STATUS:ERR_LEN\r\n");
        }
        return;
    }

    /* 4. Аутентификация: "AUTH:<пароль>" или "PASS:<пароль>" */
    if (strncmp(cmd, "AUTH:", 5) == 0 || strncmp(cmd, "PASS:", 5) == 0) {
        const char *entered = (strncmp(cmd, "AUTH:", 5) == 0) ? (cmd + 5) : (cmd + 5);
        if (s_state == STATE_BLOCKED) {
            uart_driver_write_string("STATUS:BLOCKED\r\nDISPLAY:BLOCKED\r\n");
            return;
        }

        if (strcasecmp(entered, s_master_password) == 0) {
            s_state = STATE_UNLOCKED;
            s_state_start_tick = HAL_GetTick();
            s_wrong_count = 0;
            s_input_len = 0;
            led_stop_effects();
            led_set(LED_GREEN, LED_STATE_ON);
            uart_driver_write_string("STATUS:OK\r\n");
            uart_driver_write_string("DISPLAY:OPENED\r\n");
        } else {
```

```
            s_wrong_count++;
            s_input_len = 0;
            memset(s_input_buffer, 0, sizeof(s_input_buffer));
            if (s_wrong_count >= MAX_WRONG_ATTEMPTS) {
                s_state = STATE_BLOCKED;
                s_state_start_tick = HAL_GetTick();
                led_start_blink(LED_RED, 200, BLOCK_DURATION_MS);
                uart_driver_write_string("STATUS:BLOCKED\r\n");
                uart_driver_write_string("DISPLAY:BLOCKED\r\n");
            } else {
                s_state = STATE_WRONG_FLASH;
                s_state_start_tick = HAL_GetTick();
                led_trigger_flash(LED_RED, 500);
                uart_driver_write_string("STATUS:ERR\r\n");
                uart_driver_write_string("DISPLAY:ERROR\r\n");
            }
        }
        return;
    }

    /* 5. Режим блокировки: никакие дальнейшие действия не обрабатываются */
    if (s_state == STATE_BLOCKED) {
        uart_driver_write_string("STATUS:BLOCKED\r\n");
        return;
    }

    /* 6. Интерактивная смена пароля по символу '+' */
    if (strcmp(cmd, "+") == 0) {
        s_state = STATE_SET_NEW_PASSWORD;
        s_new_pass_len = 0;
        memset(s_new_pass_buffer, 0, sizeof(s_new_pass_buffer));
        uart_driver_write_string("STATUS:ENTER_NEW_PASS\r\n");
        return;
    }

    /* 7. Подтверждение смены пароля (y/n) */
    if (s_state == STATE_CONFIRM_PASSWORD) {
        if (cmd[0] == 'y' || cmd[0] == 'Y') {
            strncpy(s_master_password, s_new_pass_buffer, MAX_PASSWORD_LEN);
            s_master_password[MAX_PASSWORD_LEN] = '\0';

            if (i2cMutexHandle != NULL) osMutexAcquire(i2cMutexHandle, osWaitForever);
            eeprom_save_password(s_master_password);
            if (i2cMutexHandle != NULL) osMutexRelease(i2cMutexHandle);

            s_state = STATE_INPUT;
            s_input_len = 0;
            uart_driver_write_string("STATUS:SAVED\r\n");
            uart_driver_write_string("DISPLAY:____\r\n");
        } else if (cmd[0] == 'n' || cmd[0] == 'N') {
            s_state = STATE_INPUT;
            s_input_len = 0;
            uart_driver_write_string("STATUS:CANCEL\r\n");
            uart_driver_write_string("DISPLAY:____\r\n");
        }
        return;
    }

    /* 8. Ввод нового пароля в интерактивном режиме */
    if (s_state == STATE_SET_NEW_PASSWORD) {
        size_t len = strlen(cmd);
        if (len >= 1 && len <= MAX_PASSWORD_LEN) {
            strncpy(s_new_pass_buffer, cmd, sizeof(s_new_pass_buffer) - 1);
            s_new_pass_buffer[sizeof(s_new_pass_buffer) - 1] = '\0';
            s_new_pass_len = (uint8_t)len;
            s_state = STATE_CONFIRM_PASSWORD;
            uart_driver_write_string("Make active? (y/n)\r\n");
        }
        return;
    }

    /* 9. Режим ввода пароля (STATE_INPUT) */
    if (s_state == STATE_INPUT) {
```

```
        /* Если пришел ровно 1 символ (посимвольный ввод из GUI on_key_type или терминала)
*/
        if (strlen(cmd) == 1 && isalnum((unsigned char)cmd[0])) {
            char c = cmd[0];
            s_last_char_tick = HAL_GetTick();

            uint8_t target_len = (uint8_t)strlen(s_master_password);
            char expected = (char)toupper((unsigned char)s_master_password[s_input_len]);
            char input_up = (char)toupper((unsigned char)c);

            if (input_up == expected) {
                s_input_buffer[s_input_len++] = c;
                led_trigger_flash(LED_YELLOW, 200);

                char disp_msg[32];
                snprintf(disp_msg, sizeof(disp_msg), "DISPLAY:%.*s\r\n", s_input_len,
s_input_buffer);
                uart_driver_write_string(disp_msg);

                if (s_input_len >= target_len) {
                    s_state = STATE_UNLOCKED;
                    s_state_start_tick = HAL_GetTick();
                    s_wrong_count = 0;
                    s_input_len = 0;
                    led_stop_effects();
                    led_set(LED_GREEN, LED_STATE_ON);
                    uart_driver_write_string("STATUS:OK\r\n");
                    uart_driver_write_string("DISPLAY:OPENED\r\n");
                }
            } else {
                s_wrong_count++;
                s_input_len = 0;
                memset(s_input_buffer, 0, sizeof(s_input_buffer));
                if (s_wrong_count >= MAX_WRONG_ATTEMPTS) {
                    s_state = STATE_BLOCKED;
                    s_state_start_tick = HAL_GetTick();
                    led_start_blink(LED_RED, 200, BLOCK_DURATION_MS);
                    uart_driver_write_string("STATUS:BLOCKED\r\n");
                    uart_driver_write_string("DISPLAY:BLOCKED\r\n");
                } else {
                    s_state = STATE_WRONG_FLASH;
                    s_state_start_tick = HAL_GetTick();
                    led_trigger_flash(LED_RED, 500);
                    uart_driver_write_string("STATUS:ERR\r\n");
                    uart_driver_write_string("DISPLAY:ERROR\r\n");
                }
            }
            return;
        }

        /* Если пришла целая строка (например в терминале ввели "1234" и нажали Enter) */
        if (strlen(cmd) > 1) {
            if (strcasecmp(cmd, s_master_password) == 0) {
                s_state = STATE_UNLOCKED;
                s_state_start_tick = HAL_GetTick();
                s_wrong_count = 0;
                s_input_len = 0;
                led_stop_effects();
                led_set(LED_GREEN, LED_STATE_ON);
                uart_driver_write_string("STATUS:OK\r\n");
                uart_driver_write_string("DISPLAY:OPENED\r\n");
            } else {
                s_wrong_count++;
                s_input_len = 0;
                memset(s_input_buffer, 0, sizeof(s_input_buffer));
                if (s_wrong_count >= MAX_WRONG_ATTEMPTS) {
                    s_state = STATE_BLOCKED;
                    s_state_start_tick = HAL_GetTick();
                    led_start_blink(LED_RED, 200, BLOCK_DURATION_MS);
                    uart_driver_write_string("STATUS:BLOCKED\r\n");
                    uart_driver_write_string("DISPLAY:BLOCKED\r\n");
                } else {
                    s_state = STATE_WRONG_FLASH;
```

```
                    s_state_start_tick = HAL_GetTick();
                    led_trigger_flash(LED_RED, 500);
                    uart_driver_write_string("STATUS:ERR\r\n");
                    uart_driver_write_string("DISPLAY:ERROR\r\n");
                }
            }
            return;
        }
    }
}

void lock_fsm_process_char(char c)
{
    if (c == '\r' || c == '\n') {
        if (s_uart_cmd_len > 0) {
            s_uart_cmd_buf[s_uart_cmd_len] = '\0';
            handle_command(s_uart_cmd_buf);
            s_uart_cmd_len = 0;
        }
        return;
    }

    if (s_uart_cmd_len < sizeof(s_uart_cmd_buf) - 1) {
        s_uart_cmd_buf[s_uart_cmd_len++] = c;
    }
}

void lock_fsm_update(uint32_t current_tick)
{
    led_update(current_tick);

    switch (s_state) {
        case STATE_INPUT:
            if (s_input_len > 0 && (current_tick - s_last_char_tick >= INPUT_TIMEOUT_MS))
{
                s_input_len = 0;
                memset(s_input_buffer, 0, sizeof(s_input_buffer));
                uart_driver_write_string("STATUS:TIMEOUT\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            }
            break;

        case STATE_UNLOCKED:
            if (current_tick - s_state_start_tick >= OPEN_DURATION_MS) {
                led_set(LED_GREEN, LED_STATE_OFF);
                s_state = STATE_INPUT;
                s_input_len = 0;
                uart_driver_write_string("STATUS:LOCKED\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            }
            break;

        case STATE_WRONG_FLASH:
            if (current_tick - s_state_start_tick >= 1000) {
                s_state = STATE_INPUT;
                uart_driver_write_string("STATUS:LOCKED\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            }
            break;

        case STATE_BLOCKED:
            if (current_tick - s_state_start_tick >= BLOCK_DURATION_MS) {
                s_wrong_count = 0;
                led_stop_effects();
                led_set(LED_RED, LED_STATE_OFF);
                s_state = STATE_INPUT;
                s_input_len = 0;
                uart_driver_write_string("STATUS:LOCKED\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            }
            break;

        default:
```

```
            break;
    }
}
```

Листинг: Core/Src/uart_driver.c

```
#include "uart_driver.h"
#include "cmsis_os.h"

extern osMessageQueueId_t uartQueueHandle;

static UART_HandleTypeDef *s_huart = NULL;

void uart_driver_init(UART_HandleTypeDef *huart)
{
    s_huart = huart;

    if (s_huart != NULL) {
        /* Сбрасываем флаг RXNE чтением регистра DR */
        __HAL_UART_FLUSH_DRREGISTER(s_huart);

        /* Включаем прерывание по приёму байта (RXNE) */
        __HAL_UART_ENABLE_IT(s_huart, UART_IT_RXNE);
        NVIC_EnableIRQ(USART6_IRQn);
    }
}

void uart_driver_write_char(char c)
{
    if (s_huart == NULL) return;

    /* Ожидание готовности передатчика (TXE) */
    while (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_TXE) == RESET) {
    }
    s_huart->Instance->DR = (uint16_t)c;
}

void uart_driver_write_string(const char *str)
{
    if (str == NULL) return;
    while (*str) {
        uart_driver_write_char(*str++);
    }
}

void uart_driver_irq_handler(void)
{
    if (s_huart == NULL) return;

    /* Проверяем флаг поступления байта RXNE */
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_RXNE) != RESET &&
        __HAL_UART_GET_IT_SOURCE(s_huart, UART_IT_RXNE) != RESET)
    {
        uint8_t received_byte = (uint8_t)(s_huart->Instance->DR & 0xFF);

        /* Отправляем байт в очередь FreeRTOS прямо из прерывания ISR */
        if (uartQueueHandle != NULL) {
            osMessageQueuePut(uartQueueHandle, &received_byte, 0, 0);
        }
    }

    /* Очистка флагов ошибок (Overrun, Noise, Framing), если возникли */
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_ORE) != RESET) {
        __HAL_UART_CLEAR_OREFLAG(s_huart);
    }
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_NE) != RESET) {
        __HAL_UART_CLEAR_NEFLAG(s_huart);
    }
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_FE) != RESET) {
        __HAL_UART_CLEAR_FEFLAG(s_huart);
    }
```

```
}
```

Листинг: Core/Src/main.c (Задачи FreeRTOS)

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body (FreeRTOS Code Lock)
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "cmsis_os.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "uart_driver.h"
#include "led.h"
#include "oled.h"
#include "eeprom.h"
#include "lock_fsm.h"
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
I2C_HandleTypeDef hi2c1;

TIM_HandleTypeDef htim1;
TIM_HandleTypeDef htim4;

UART_HandleTypeDef huart6;

/* Definitions for displayTask */
osThreadId_t displayTaskHandle;
const osThreadAttr_t displayTask_attributes = {
  .name = "displayTask",
  .stack_size = 512 * 4,
  .priority = (osPriority_t) osPriorityNormal,
};
/* Definitions for lockTask */
osThreadId_t lockTaskHandle;
const osThreadAttr_t lockTask_attributes = {
  .name = "lockTask",
  .stack_size = 512 * 4,
  .priority = (osPriority_t) osPriorityAboveNormal,
};
/* Definitions for uartQueue */
osMessageQueueId_t uartQueueHandle;
const osMessageQueueAttr_t uartQueue_attributes = {
  .name = "uartQueue"
};

/* Мьютекс для защиты шины I2C1 (разделяемой между SSD1306 и MCP79411) */
```

```
osMutexId_t i2cMutexHandle;
const osMutexAttr_t i2cMutex_attributes = {
  .name = "i2cMutex"
};

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_TIM1_Init(void);
static void MX_TIM4_Init(void);
static void MX_USART6_UART_Init(void);
void StartDisplayTask(void *argument);
void StartLockTask(void *argument);

/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* MCU Configuration--------------------------------------------------------*/
  HAL_Init();

  /* Configure the system clock */
  SystemClock_Config();

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_I2C1_Init();
  MX_TIM1_Init();
  MX_TIM4_Init();
  MX_USART6_UART_Init();

  /* USER CODE BEGIN 2 */
  uart_driver_init(&huart6);
  led_init();
  eeprom_init(&hi2c1);
  /* USER CODE END 2 */

  /* Init scheduler */
  osKernelInitialize();

  /* Create the mutex(es) */
  i2cMutexHandle = osMutexNew(&i2cMutex_attributes);

  /* Create the queue(s) */
  uartQueueHandle = osMessageQueueNew(64, sizeof(uint8_t), &uartQueue_attributes);

  /* Create the thread(s) */
  displayTaskHandle = osThreadNew(StartDisplayTask, NULL, &displayTask_attributes);
  lockTaskHandle = osThreadNew(StartLockTask, NULL, &lockTask_attributes);

  /* Start scheduler */
  osKernelStart();

  /* We should never get here as control is now taken by the scheduler */
  while (1)
  {
  }
```

```
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 15;
  RCC_OscInitStruct.PLL.PLLN = 72;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 4;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
  {
    Error_Handler();
  }
}

static void MX_I2C1_Init(void)
{
  hi2c1.Instance = I2C1;
  hi2c1.Init.ClockSpeed = 400000;
  hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
  hi2c1.Init.OwnAddress1 = 0;
  hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
  hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c1.Init.OwnAddress2 = 0;
  hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
  if (HAL_I2C_Init(&hi2c1) != HAL_OK)
  {
    Error_Handler();
  }

  if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
  {
    Error_Handler();
  }
}

static void MX_TIM1_Init(void)
{
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};

  htim1.Instance = TIM1;
```

```
  htim1.Init.Prescaler = 59;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 999;
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim1.Init.RepetitionCounter = 0;
  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
  sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
  if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
  sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
  sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
  sBreakDeadTimeConfig.DeadTime = 0;
  sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
  sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
  sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
  if (HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig) != HAL_OK)
  {
    Error_Handler();
  }
  HAL_TIM_MspPostInit(&htim1);
}

static void MX_TIM4_Init(void)
{
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  htim4.Instance = TIM4;
  htim4.Init.Prescaler = 59;
  htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim4.Init.Period = 1999;
  htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim4) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim4, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
  {
    Error_Handler();
```

```
  }
  if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_4) != HAL_OK)
  {
    Error_Handler();
  }
  HAL_TIM_MspPostInit(&htim4);
}

static void MX_USART6_UART_Init(void)
{
  huart6.Instance = USART6;
  huart6.Init.BaudRate = 9600;
  huart6.Init.WordLength = UART_WORDLENGTH_8B;
  huart6.Init.StopBits = UART_STOPBITS_1;
  huart6.Init.Parity = UART_PARITY_NONE;
  huart6.Init.Mode = UART_MODE_TX_RX;
  huart6.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart6.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart6) != HAL_OK)
  {
    Error_Handler();
  }
}

static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOE_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin : PC15 */
  GPIO_InitStruct.Pin = GPIO_PIN_15;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
}

/* ========================================================================== */
/*                              ЗАДАЧИ FREERTOS                               */
/* ========================================================================== */

/**
  * @brief  Задача 1: Отрисовка OLED дисплея с динамическим счетчиком FPS
  *         Требование методички: не менее 20 кадров/сек (период <= 50 мс)
  *         и счетчик 00..99 в углу экрана.
  */
void StartDisplayTask(void *argument)
{
  /* Инициализация OLED под защитой мьютекса */
  if (i2cMutexHandle != NULL) osMutexAcquire(i2cMutexHandle, osWaitForever);
  oled_Init();
  if (i2cMutexHandle != NULL) osMutexRelease(i2cMutexHandle);

  uint8_t fps_counter = 0;
  LockDisplayInfo disp_info;

  for(;;)
  {
    /* 1. Считываем снимок данных из модуля замка */
    lock_fsm_get_display_info(&disp_info);

    /* 2. Отрисовываем кадр в буфере ОЗУ (без блокировки шины I2C!) */
    oled_Fill(Black);
    oled_DrawSquare(0, 127, 0, 63, White);

    /* Заголовок */
    oled_SetCursor(8, 5);
    oled_WriteString("CODE LOCK", Font_11x18, White);
```

```
    /* Динамический счетчик активности в правом верхнем углу (00..99) */
    char fps_str[6];
    snprintf(fps_str, sizeof(fps_str), "%02u", fps_counter);
    oled_SetCursor(108, 5);
    oled_WriteString(fps_str, Font_7x10, White);

    fps_counter = (fps_counter + 1) % 100;

    switch (disp_info.state) {
        case STATE_INPUT:
            oled_SetCursor(10, 26);
            oled_WriteString("Status: LOCKED", Font_7x10, White);
            oled_SetCursor(20, 44);
            oled_WriteString(disp_info.input_mask, Font_11x18, White);
            break;

        case STATE_UNLOCKED:
            oled_SetCursor(10, 26);
            oled_WriteString(">> OPENED! <<", Font_7x10, White);
            oled_SetCursor(15, 44);
            oled_WriteString("ACCESS OK", Font_11x18, White);
            break;

        case STATE_WRONG_FLASH:
            oled_SetCursor(10, 26);
            oled_WriteString(">> ERROR! <<", Font_7x10, White);
            oled_SetCursor(15, 44);
            oled_WriteString("WRONG CODE", Font_11x18, White);
            break;

        case STATE_BLOCKED: {
            oled_SetCursor(10, 26);
            oled_WriteString("!! BLOCKED !!", Font_7x10, White);
            char b_buf[16];
            snprintf(b_buf, sizeof(b_buf), "Wait: %lu s", disp_info.block_rem_sec);
            oled_SetCursor(25, 44);
            oled_WriteString(b_buf, Font_7x10, White);
            break;
        }

        case STATE_SET_NEW_PASSWORD:
            oled_SetCursor(10, 26);
            oled_WriteString("NEW PASS:", Font_7x10, White);
            oled_SetCursor(20, 44);
            oled_WriteString(disp_info.new_pass, Font_11x18, White);
            break;

        case STATE_CONFIRM_PASSWORD:
            oled_SetCursor(10, 26);
            oled_WriteString("Save to EEPROM?", Font_7x10, White);
            oled_SetCursor(20, 44);
            oled_WriteString("Press 'y' / 'n'", Font_7x10, White);
            break;
    }

    /* 3. Захватываем мьютекс ТОЛЬКО на время передачи буфера по шине I2C */
    if (i2cMutexHandle != NULL) osMutexAcquire(i2cMutexHandle, osWaitForever);
    oled_UpdateScreen();
    if (i2cMutexHandle != NULL) osMutexRelease(i2cMutexHandle);

    /* Задержка 40 мс = ровно 25 кадров в секунду (>= 20 FPS по ТЗ!) */
    osDelay(40);
  }
}

/**
  * @brief  Задача 2: Логика кодового замка и обработка команд
  */
void StartLockTask(void *argument)
{
  lock_fsm_init();
```

```
  uint8_t rx_char;

  for(;;)
  {
    /* 1. Извлекаем поступившие символы из очереди FreeRTOS (таймаут 10 мс) */
    if (osMessageQueueGet(uartQueueHandle, &rx_char, NULL, 10) == osOK) {
        lock_fsm_process_char((char)rx_char);
    }

    /* 2. Обновляем конечный автомат замка и анимацию светодиодов */
    lock_fsm_update(HAL_GetTick());
  }
}

/**
  * @brief  Period elapsed callback in non blocking mode (TIM6 HAL Tick)
  */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM6) {
    HAL_IncTick();
  }
}

void Error_Handler(void)
{
  __disable_irq();
  while (1)
  {
  }
}
```